
<Company Name>

<Project Name>
Software Architecture Document

Version <1.0>

[The texts in blue are guidance for filling in the information for each section. Remove everything in blue when writing the document.]

This template is a simplified version of the Software Architecture Document from the RUP model.]

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

Revision History

Date	Version	Description	Author
<dd/mmm/yy>	<x.x>	<details>	<name>

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

Table of Contents

1.	Introduction	4
2.	Architectural Goals and Constraints	4
3.	Use-Case Model	4
4.	Logical View	4
4.1	Component: abc	4
4.2	Component: Database	4
4.2.1	Responsibilities	4
4.2.2	Technology and Physical Design	5
4.2.3	Entity-Relationship Models	5
4.2.4	Entity Catalog	6
4.2.5	Key Relationships and Lifecycle Rules	7
4.2.6	Integrity Constraints	8
4.2.7	Indexes and Query Support	9
4.2.8	Security and Data Handling	9
5.	Deployment	9
6.	Implementation View	9

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

Software Architecture Document

1. Introduction

*[The introduction of the **Software Architecture Document** provides an overview of the entire **Software Architecture Document**. It includes the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the **Software Architecture Document**.]*

2. Architectural Goals and Constraints

[This section describes the software requirements and objectives that have some significant impact on the architecture; for example, safety, security, privacy, use of an off-the-shelf product, portability, distribution, and reuse. It also captures the special constraints that may apply: design and implementation strategy, development tools, team structure, schedule, legacy code, and so on.]

Vietnamese: Phần này nêu các mục tiêu và ràng buộc của kiến trúc phần mềm. Các mục tiêu và ràng buộc này lấy từ yêu cầu phi chức năng trong tài liệu Vision. Ví dụ, các ràng buộc/mục tiêu như yêu cầu áp dụng ngôn ngữ lập trình, môi trường của ứng dụng (như web, mobile, hay cả hai), ràng buộc về bảo mật, hiệu năng, v.v...

Các bạn chỉ cần gạch đầu dòng các mục tiêu và ràng buộc quan trọng ở đây thôi.]

3. Use-Case Model

[This section includes the use case diagrams that are already modeled and presented in the use-case specification document.]

4. Logical View

[This section describes the architecture with components and relationships among them. One or several diagrams showing the architecture are provided here. For each component, describe its responsibilities and/or services that are provided for other components. Each relationship should also indicate the means of communication, such as HTTP, HTTPS, Socket, LAN, Internet, etc.]

The detail of each component is provided using the subsection below.]

4.1 Component: abc

[This section provides details for the component named “abc”. You need to include class diagrams for this component and explain key classes.]

For each component, create a section like this.]

4.2 Component: Database

4.2.1 Responsibilities

The Database component provides durable, relational storage for FitCV. It preserves user and organization data, CV versions, job and job-description data, applications, AI parsing outputs, match evidence, improvement suggestions, and asynchronous AI task state. The component is accessed by the FastAPI backend through SQLAlchemy repositories; the React frontend does not connect to the database directly.

- Maintain authentication identities and role-aware links to companies and candidates.
- Preserve uploaded CV metadata and immutable version history while keeping the binary file in external file storage.
- Store recruitment jobs, HR ownership, candidate applications, student application tracking, reminders, notes, and status history.

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

- Persist AI input/output provenance so every match score and suggestion can be traced to the exact CV, JD, parser output, and algorithm version.

4.2.2 *Technology and Physical Design*

The physical schema targets MySQL 8.0 or later. All tables use the InnoDB engine for transactions and foreign-key enforcement, and utf8mb4 with utf8mb4_unicode_ci for multilingual CV and JD text. Primary identifiers use BIGINT UNSIGNED AUTO_INCREMENT unless a bridge table requires a composite key. Timestamps use DATETIME values generated by the database where appropriate.

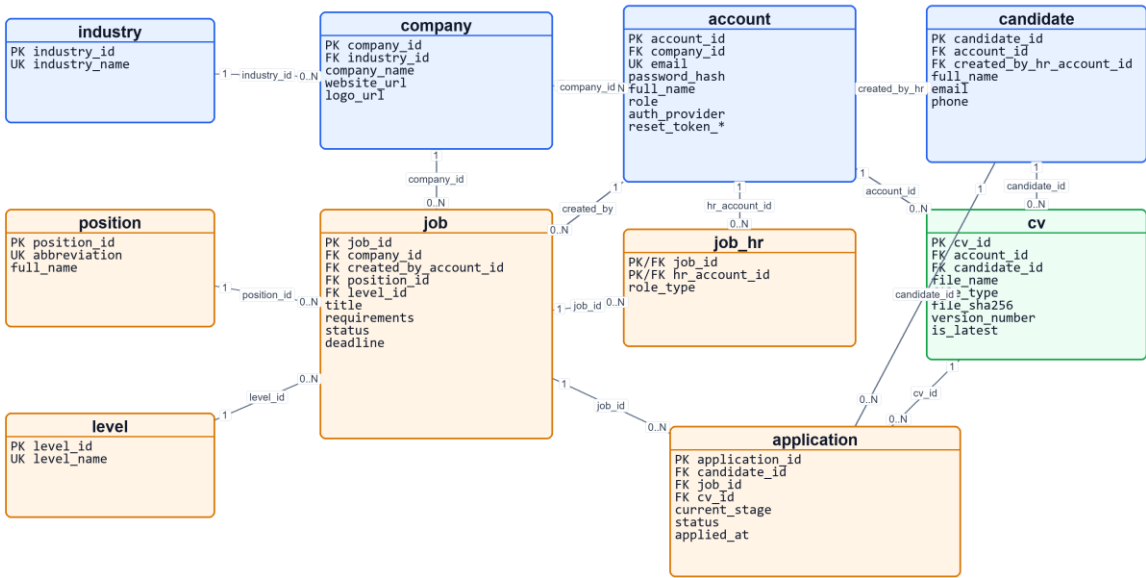
- Application access path: React/Vite → HTTPS REST API → FastAPI service → repository → SQLAlchemy → MySQL.
- Large CV files are stored outside MySQL; the cv table stores the path, MIME type, size, SHA-256 hash, and version metadata.
- Structured AI payloads and evidence use JSON columns so the schema can retain explainable model output without losing relational ownership and lifecycle rules.

4.2.3 *Entity-Relationship Models*

The ER design is split into two readable views of one physical schema. Solid lines represent enforced foreign keys. The dashed ai_task link is logical because ai_task uses the generic pair task_type and resource_id rather than a physical foreign key.

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

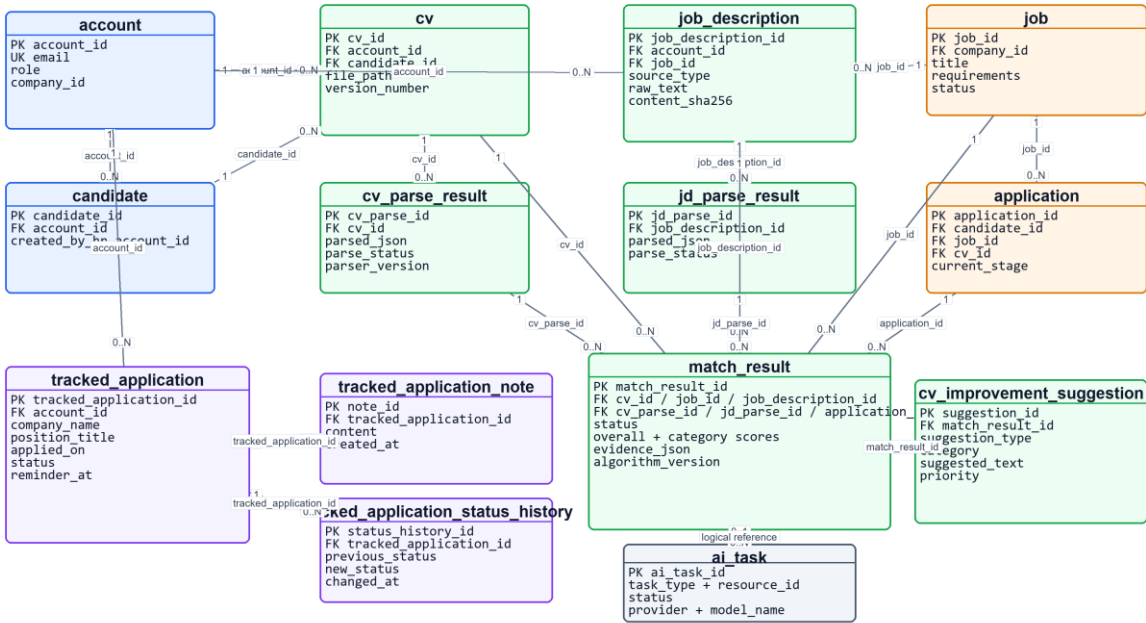
FitCV ER Model A - Identity, Recruitment, CV, and Hiring Pipeline



Notation: PK = primary key, FK = foreign key, UK = unique key. Cardinalities reflect the physical FitCV schema.

Figure 1. FitCV ER model A — identity, recruitment, CV ownership, and the hiring pipeline.

FitCV ER Model B - CV/JD Analysis, AI Results, and Student Application Tracking



Notation: PK = primary key, FK = foreign key, UK = unique key. Cardinalities reflect the physical FitCV schema.

Figure 2. FitCV ER model B — parsing, matching, AI suggestions, and student application tracking.

4.2.4 Entity Catalog

Table 1 summarizes every physical table in database/full_schema.sql. The grouping is architectural only; all entities belong to the same FitCV MySQL schema.

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

Domain	Entity	Primary responsibility
Identity & organization	industry	Industry reference data used to classify companies.
Identity & organization	company	Employer profile linked to an industry and to HR accounts and jobs.
Identity & organization	account	Authentication identity, role, profile, company link, and password-reset state.
Identity & organization	position	Normalized job-position reference data.
Identity & organization	level	Normalized seniority-level reference data.
CV & candidate	candidate	Candidate profile for self-service students or HR-created applicants.
CV & candidate	cv	Uploaded CV metadata, ownership, SHA-256 hash, and version history.
CV & candidate	cv_parse_result	Structured output and status produced by the CV parser.
Recruitment	job	Job post, company, owner, position, level, requirements, status, and deadline.
Recruitment	job_hr	Many-to-many assignment of HR accounts to jobs with a role type.
Recruitment	job_description	Reusable or job-linked JD input with raw text and content hash.
Recruitment	jd_parse_result	Structured requirements extracted from a job description.
Recruitment	application	Candidate-to-job application using a selected CV and current hiring stage.
Student tracking	tracked_application	Student-managed external application, status, follow-up, and reminder.
Student tracking	tracked_application_note	Time-ordered notes belonging to a tracked application.
Student tracking	tracked_application_status_history	Audit history for every tracked-application status change.
AI analysis	match_result	CV/JD score, category scores, pass probability, evidence, and algorithm version.
AI analysis	cv_improvement_suggestion	Prioritized AI suggestions derived from one match result.
AI orchestration	ai_task	Asynchronous AI work item with provider, model, status, attempts, and error data.

4.2.5 Key Relationships and Lifecycle Rules

Table 2 records the principal cardinalities and deletion behavior. CASCADE is used for true dependent records, SET NULL preserves optional historical context, and RESTRICT prevents deletion when a referenced record must remain valid.

Relationship	Foreign key	On delete	Design meaning
industry 1 — 0..N company	company.industry_id	SET NULL	An industry can classify many companies.

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

Relationship	Foreign key	On delete	Design meaning
company 1 — 0..N account	account.company_id	SET NULL	A company can be linked to many internal users.
account 1 — 0..1 candidate	candidate.account_id	SET NULL	A student account may own one candidate profile.
account 1 — 0..N candidate	candidate.created_by_hr_account_id	SET NULL	HR can create multiple candidate profiles.
account/candidate 1 — 0..N cv	cv.account_id or cv.candidate_id	CASCADE	Every CV has at least one owner path.
cv 1 — 0..N cv_parse_result	cv_parse_result.cv_id	CASCADE	A CV may be parsed repeatedly across parser versions.
company 1 — 0..N job	job.company_id	CASCADE	Deleting a company removes its jobs.
account 1 — 0..N job	job.created_by_account_id	RESTRICT	A job retains a valid creator.
job N — N account	job_hr(job_id, hr_account_id)	CASCADE	HR assignment is resolved by a composite-key bridge.
account/job 1 — 0..N job_description	job_description.account_id /job_id	CASCADE/ SET NULL	A JD belongs to an account and may reference a job.
job_description 1 — 0..N jd_parse_result	jd_parse_result.job_description_id	CASCADE	A JD may have multiple parse attempts or versions.
candidate/job/cv 1 — 0..N application	application candidate/job/cv FKs	CASCADE/ RESTRICT	The application stores the exact CV used.
account 1 — 0..N tracked_application	tracked_application.account_id	CASCADE	Student tracking data is account-scoped.
tracked_application 1 — 0..N note/history	tracked_application_id	CASCADE	Notes and audit history follow the tracked application.
CV/JD inputs 1 — 0..N match_result	match_result source FKs	CASCADE	A score remains traceable to the inputs and parser outputs.
application 1 — 0..N match_result	match_result.application_id	SET NULL	A match may support a formal application.
match_result 1 — 0..N suggestion	cv_improvement_suggestion.match_result_id	CASCADE	Suggestions cannot outlive their source analysis.

4.2.6 Integrity Constraints

- A CV must belong to an account or a candidate: account_id IS NOT NULL OR candidate_id IS NOT NULL.
- CV version_number is unique per account, allowing reliable history and latest-version retrieval.
- Every match_result must reference either a job or a standalone job_description.
- Overall, skill, experience, education, soft-skill, and pass-probability values are constrained to the inclusive range 0–100.
- The tuple (cv_parse_id, jd_parse_id, algorithm_version) is unique, preventing duplicate results for the same parsed inputs and scoring algorithm.
- Reference names and identities are protected by unique keys, including industry_name, position.abbreviation, level_name, and account.email.

<Project Name>	Version: <1.0>
Software Architecture Document	Date: <dd/mmm/yy>
<document identifier>	

4.2.7 Indexes and Query Support

Foreign-key and workflow indexes support the dominant FitCV access patterns: accounts by company or role; candidate ownership; latest CV version; parser output by CV or JD; jobs by company or creator; applications by candidate or job; tracked applications by account, date, status, or reminder; match history by CV and generation time; and ordered suggestions by match and type. `ai_task` is indexed by `task_type`, `resource_id`, and `created_at` for background-worker polling and audit lookup.

4.2.8 Security and Data Handling

- Passwords and password-reset codes are stored only as hashes; reset expiry is stored separately.
- Database credentials are supplied through environment variables and are never committed to source control.
- Backend authorization restricts data by authenticated account and role before repository queries are executed.
- SHA-256 values support duplicate detection and traceability for CV files and JD text; they are not a replacement for access control.
- AI scores are decision-support information, not guaranteed hiring outcomes; `evidence_json` and `algorithm_version` support review and reproducibility.

5. Deployment

[Leave this section blank for PA3 if you are writing this document for PA4.]

In this section, describe how the system is deployed by mapping the components in Section 4 to machines running them. For example, your mobile app is running on a mobile device (Android, iOS, etc), your server runs all components on the server side including the database]

6. Implementation View

[Leave this section blank for PA3 if you are writing this document for PA4.]

In this section, provide folder structures for your code for all components described in Section 4.]